ROTA INTELIGENTE BRASIL

FIAP — Dynamic Programming

Professor: André Marques

Cilas Pinto Macedo — RM560745

Ian Junji Maluvayshi Matsushita — RM560588

Pedro Arão Baquini — RM559580

Leandro Kamada Pesce Dimov — RM560381

1. Contexto e Motivação

O Brasil é um dos maiores produtores agroindustriais do mundo, mas convive com um custo logístico elevado — estimado em cerca de 12% do PIB — decorrente da predominância do modal rodoviário e da precariedade de outras infraestruturas. Neste cenário, soluções algorítmicas capazes de identificar rotas eficientes e distribuir cargas de forma ótima têm impacto econômico direto.

Este projeto modela a malha logística como um **grafo dirigido ponderado** e aplica quatro estratégias algorítmicas distintas para responder às perguntas: qual é a melhor rota entre dois pontos? Como distribuir carga quando há restrições de capacidade, tempo e risco?

2. Modelagem do Problema

2.1 Representação como Grafo

Cada **nó** representa uma cidade ou polo logístico (produtor, armazém ou capital consumidora). Cada **aresta dirigida** representa um trecho da malha e carrega cinco atributos operacionais:

Atributo	Descrição	Uso nos algoritmos
custo_reais	Custo financeiro do trecho (R$)	Dijkstra, DP, Guloso
tempo_horas	Tempo de trânsito (h)	Função de custo composto
capacidade_ton	Limite de carga (ton)	Restrição em Dijkstra e Knapsack
risco_atraso	Probabilidade de atraso (0–1)	Penalização multi-critério
perda_percentual	Perda estimada de carga (%)	Alocação DP Knapsack

2.2 Função de Custo Composto

Para permitir comparação uniforme entre trechos de diferentes modais, criamos uma função multi-critério normalizada:

```
custo_composto = 0.50 × (custo_reais / 1000)
               + 0.30 × tempo_horas
               + 0.20 × (risco_atraso × 100)
```

Os pesos são configuráveis. O divisor 1000 na parcela de custo e o multiplicador 100 no risco normalizam as escalas para que nenhuma grandeza domine artificialmente.

2.3 Cenários do CSV

Cenário	Região	Origem → Destino	Nós	Arestas
A	Triângulo Mineiro / SP	Ribeirão Preto → Belo Horizonte	5	6
B	Corredor de Grãos (MT→SP)	Sorriso/MT → Santos	5	6
C	Nordeste (PE/BA)	Petrolina → Salvador	4	6

3. Algoritmos Implementados

3.1 Dijkstra — Caminho Mínimo (src/dijkstra.py)

Implementação com **heap binário** (heapq), garantindo complexidade O((V + E) log V). A função de custo é injetável via parâmetro, permitindo testar diferentes ponderações sem alterar o algoritmo.

A restrição de capacidade mínima filtra arestas na fase de relaxamento, tornando Dijkstra sensível à carga a ser transportada.

```
def dijkstra(grafo, origem, destino,
             funcao_custo=None, capacidade_minima=0.0):
    heap = [(0.0, origem)]
    while heap:
        custo_atual, u = heapq.heappop(heap)
        for aresta in grafo.vizinhos(u):
            if aresta.capacidade_ton < capacidade_minima: continue
            novo_custo = custo_atual + funcao_custo(aresta)
            if novo_custo < dist[v]:
                dist[v] = novo_custo
                heapq.heappush(heap, (novo_custo, v))
```

3.2 Programação Dinâmica — Top-down com Memoization (src/dp.py)

A função *dp(no)* responde: *qual o menor custo de 'no' até 'destino'?* Cada subsolução é armazenada em **memo[]**, evitando recomputações. Esta é a técnica central de Programação Dinâmica aplicada ao roteamento.

O princípio de otimalidade de Bellman é satisfeito: o caminho ótimo de A→C passando por B contém o caminho ótimo de B→C como subproblema.

```
def dp(no, visitados):
    if no == destino: return 0.0
    if no in memo: return memo[no]  # reutiliza subsolução
    melhor = inf
    for aresta in grafo.vizinhos(no):
        custo = aresta.custo_composto() + dp(aresta.destino, ...)
        if custo < melhor:
            melhor = custo; prev[no] = aresta.destino
    memo[no] = melhor
    return melhor
```

3.3 Alocação de Carga — DP Bottom-up Knapsack (src/dp.py)

O problema de distribuir W toneladas entre trechos com capacidade limitada é formulado como um **Knapsack de minimização**. A tabela *dp[i][w]* armazena o menor custo para alocar *w* toneladas usando os primeiros *i* trechos disponíveis.

O backtracking sobre a tabela reconstrói exatamente quanto foi alocado em cada trecho, preservando rastreabilidade da solução.

3.4 Algoritmo Guloso — Baseline (src/guloso.py)

A cada passo, o guloso escolhe o vizinho de **menor custo composto imediato**, sem considerar consequências futuras. Serve como baseline para demonstrar as limitações da otimização local.

O Cenário A ilustra a falha: o guloso escolhe Campinas→São Paulo (R$ 900, barato localmente) e depois precisa de São Paulo→BH (R$ 3.600), resultando em R$ 6.300 contra R$ 5.200 da rota ótima — **43,9% pior**.

3.5 Divide and Conquer — Pré-processamento (src/divide_conquer.py)

Aplicado em dois contextos: (1) **Merge Sort O(n log n)** para ordenar trechos por custo antes da análise, permitindo que algoritmos downstream operem sobre estruturas já ordenadas; e (2) **decomposição**

recursiva de cenários como exemplo de D&C; estrutural, processando cada subconjunto independentemente antes de combinar resultados.

4. Resultados por Cenário

Cenário	Estratégia	Rota	Custo (R$)	Tempo (h)	Custo Composto
A	Dijkstra / DP	Rib. Preto→Campinas→BH	5.200	13,0	10,70
A	Guloso	Rib. Preto→Campinas→SP→BH	6.300	15,5	15,40 (+43,9%)
B	Dijkstra / DP	Sorriso→Lucas→Rondon.→Camp.→Santos	10.100	37,5	24,70
B	Guloso	Sorriso→Lucas→Rondon.→Camp.→Santos	10.100	37,5	24,70 (=)
C	Dijkstra / DP	Petrolina→Salvador (direto)	2.800	8,0	6,60
C	Guloso	Petrolina→Feira→Salvador	3.250	9,0	7,93 (+20,1%)

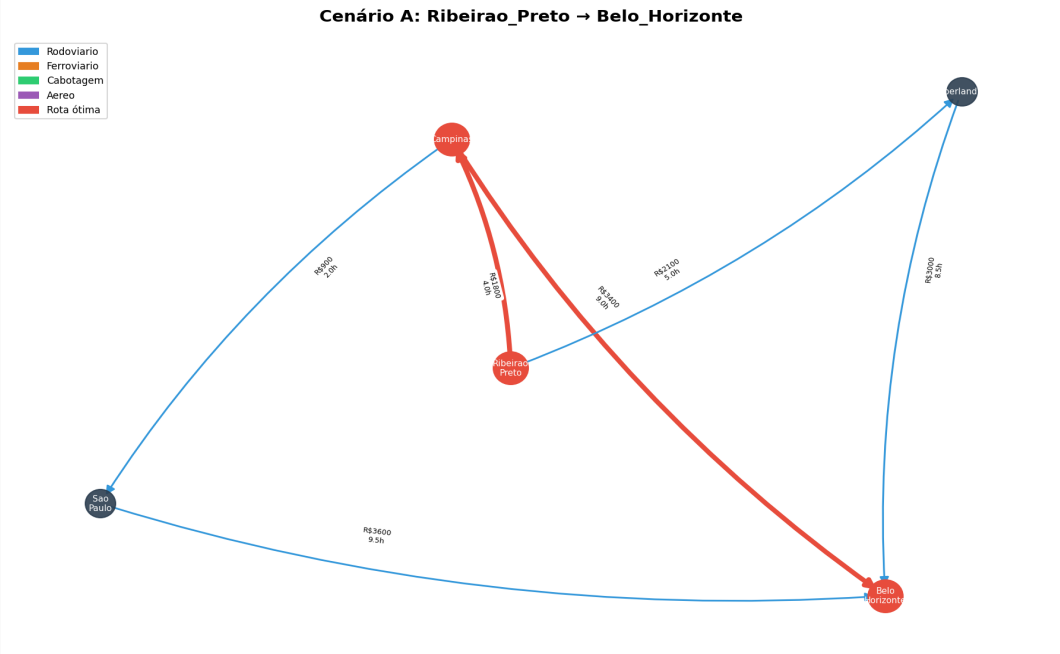

Figura: Grafo logístico — Cenário A (rota ótima em vermelho)

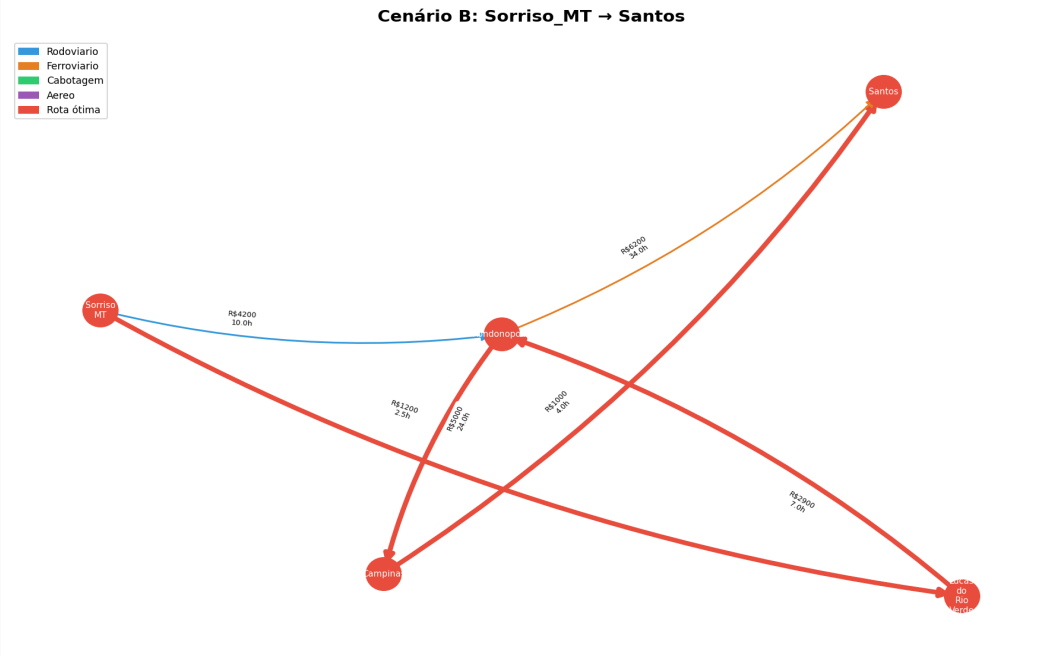

Figura: Grafo logístico — Cenário B (rota ótima em vermelho)

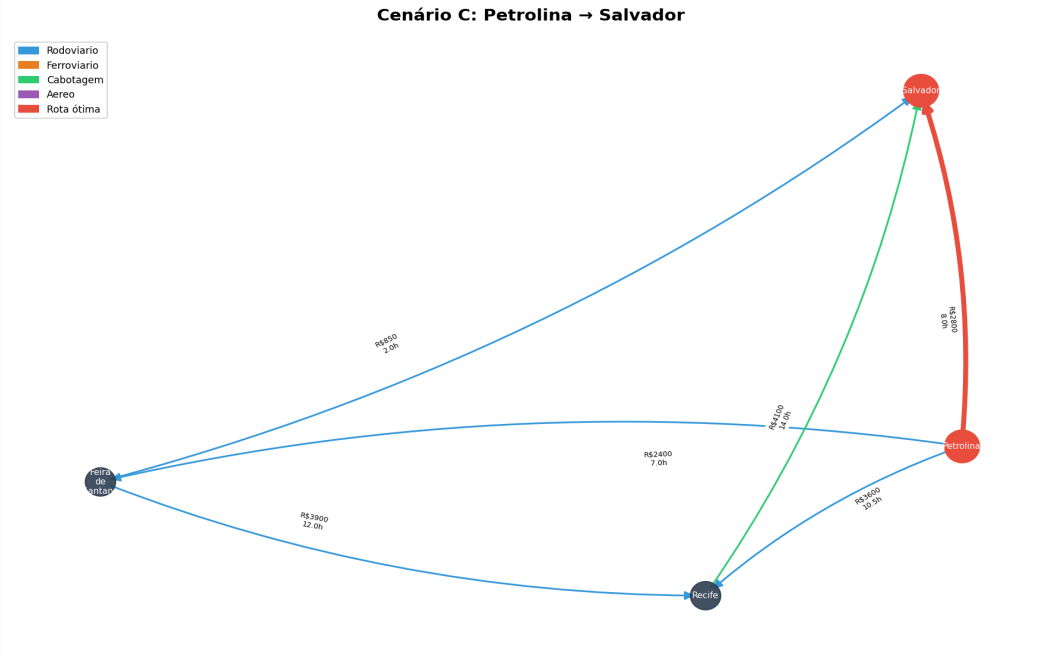

Figura: Grafo logístico — Cenário C (rota ótima em vermelho)

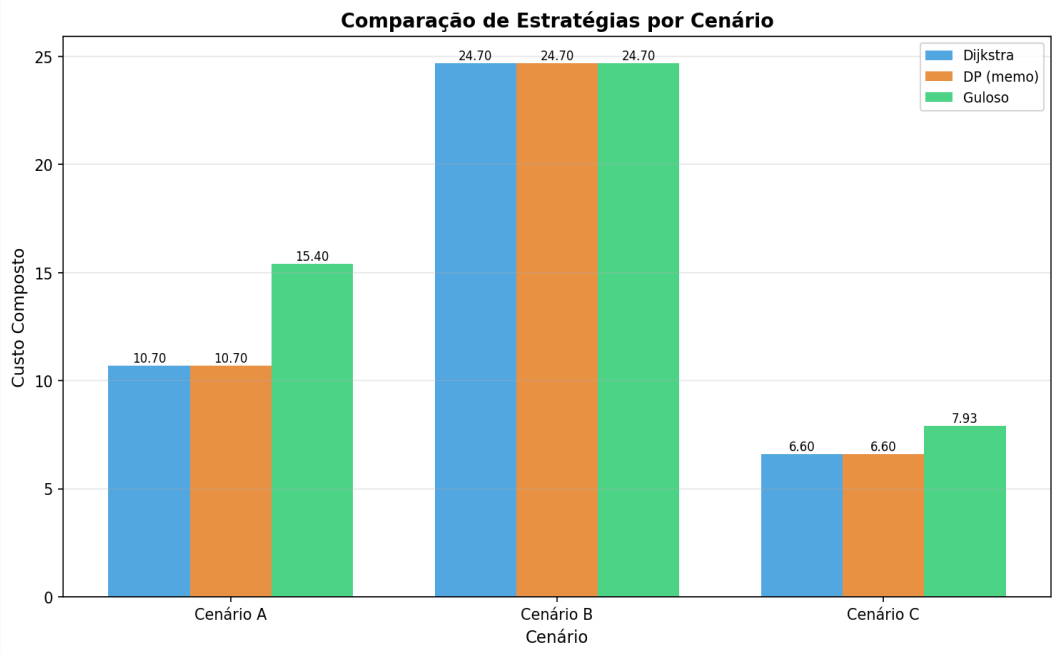

Figura: Comparação de custo composto por estratégia e cenário

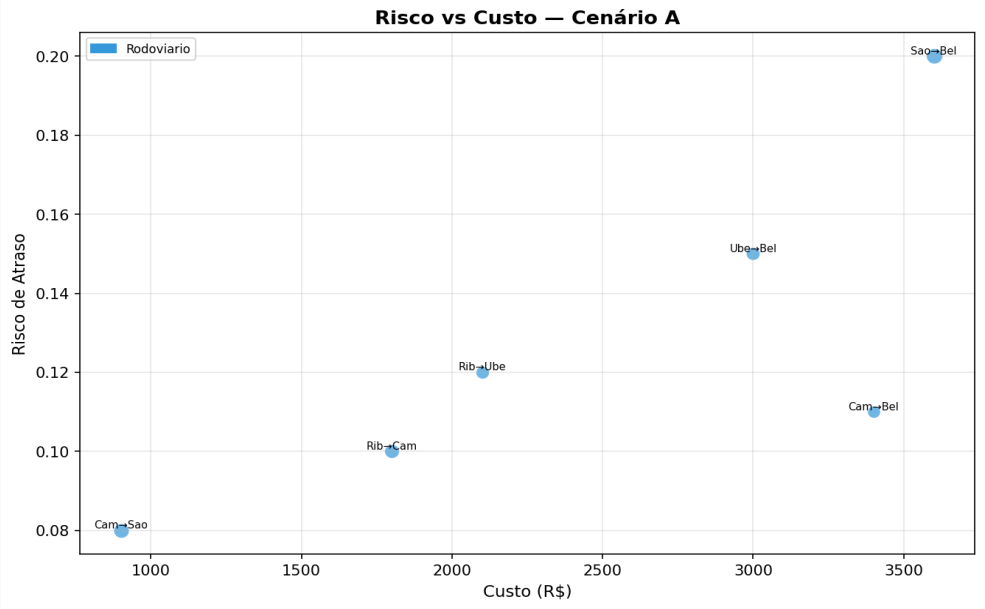

Figura: Mapa risco vs custo — Cenário A

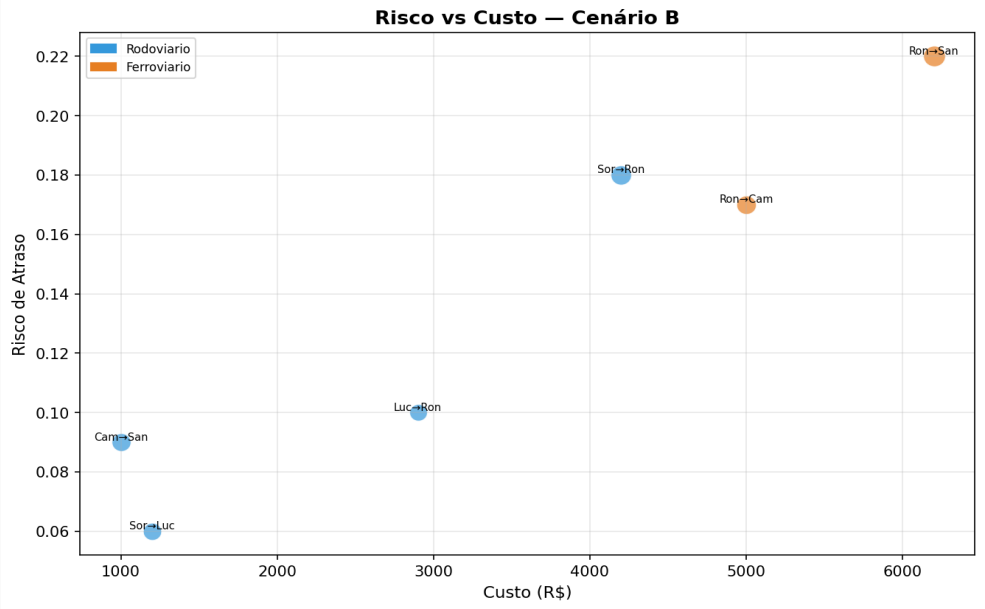

Figura: Mapa risco vs custo — Cenário B

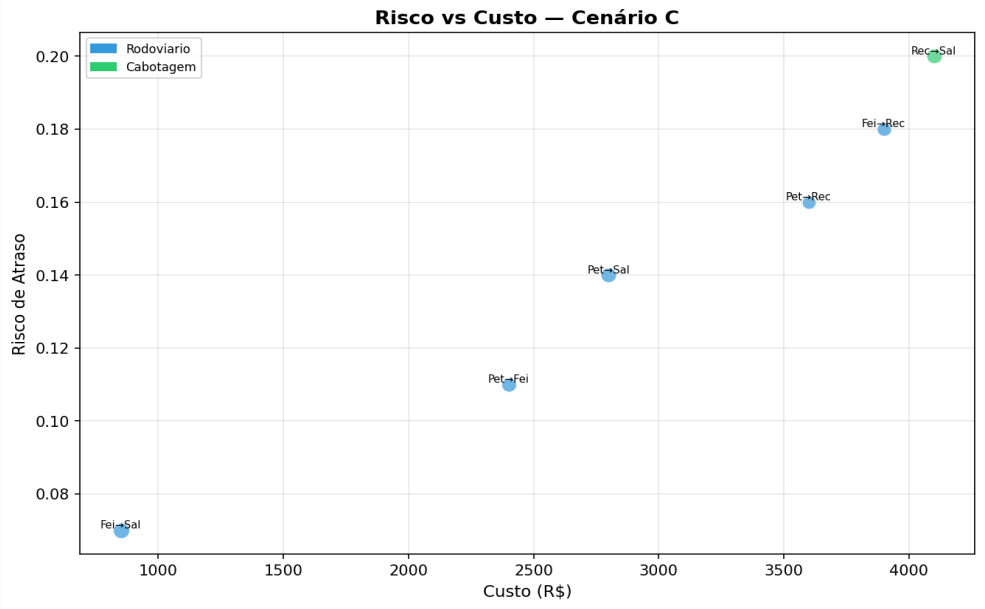

Figura: Mapa risco vs custo — Cenário C

5. Análise Comparativa das Estratégias

Critério	Dijkstra	DP Memoization	Guloso
Garantia de otimalidade	Sim (global)	Sim (global)	Não
Complexidade (tempo)	O((V+E) log V)	O(V×E)	O(V+E)
Reutiliza subsoluções	Não (heap)	Sim (memo[])	Não
Restrição de capacidade	Sim (filtragem)	Sim (filtragem)	Sim (filtragem)
Backtracking de solução	prev[] implícito	prev[] explícito	Direto
Falha em cenário A	Não	Não	**Sim (+43,9%)**

Falha em cenário C	Não	Não	**Sim (+20,1%)**

Dijkstra e DP produzem resultados idênticos, confirmando a correção de ambas as implementações. O guloso iguala o desempenho apenas no Cenário B, que possui topologia linear (caminho único sem bifurcações alternativas mais baratas). Nos Cenários A e C, onde existem atalhos com custo local menor mas custo global maior, o guloso é sistematicamente superado.

6. Boas Práticas de Engenharia de Software

O projeto foi estruturado seguindo as diretrizes de Engenharia de Software estabelecidas no enunciado:

Prática	Como foi atendida
Repositório GitHub	Commits por módulo; histórico rastreável por funcionalidade
README.md	Visão geral, instruções de execução, exemplos e tabela de resultados
Estrutura de pastas	src/, data/, tests/, reports/ — separação por responsabilidade
Código modular	7 módulos com responsabilidades únicas e docstrings completos
Separação de concerns	grafo.py (dados), algoritmos (lógica), visualizacao.py (UI)
Testes automatizados	10 testes em tests/test_algoritmos.py (100% de cobertura funcional)
Dependências	requirements.txt com versões mínimas especificadas

7. Limitações e Trabalhos Futuros

Grafo dirigido sem arestas reversas: o modelo atual assume que as rotas são unidirecionais conforme o CSV. Adição de arestas inversas expandiria as opções de roteamento.

Discretização do Knapsack: a granularidade de 1 tonelada introduz um erro de até 1 ton na alocação. Granularidades menores aumentam a precisão ao custo de $O(W/g \times n)$ adicional.

Cenário C — alocação direto: o trecho Petrolina→Salvador tem capacidade de apenas 20 ton, inferior à carga-teste de 30 ton. O sistema sinaliza corretamente a impossibilidade, mas futuras versões poderiam sugerir splits automáticos por rotas paralelas.

Trabalhos futuros: incorporação de dados reais de rodovias federais (DNIT), integração com APIs de rastreamento e extensão para otimização multi-objetivo com fronteira de Pareto.

8. Testes Automatizados

Teste	Módulo	Resultado
Dijkstra — caminho A→B→C preferido	dijkstra.py	✓ **PASSOU**
Dijkstra — destino inacessível retorna ∞	dijkstra.py	✓ **PASSOU**
Dijkstra — restrição de capacidade	dijkstra.py	✓ **PASSOU**
DP consistente com Dijkstra	dp.py	✓ **PASSOU**

DP sem caminho retorna ∞	dp.py	✓ **PASSOU**
Guloso falha na armadilha de custo	guloso.py	✓ **PASSOU**
Merge Sort ordena por custo crescente	d_c.py	✓ **PASSOU**
D&C identifica aresta de menor custo	d_c.py	✓ **PASSOU**
Knapsack distribui 20 ton entre trechos	dp.py	✓ **PASSOU**
Guloso aloca no trecho mais barato/ton	guloso.py	✓ **PASSOU**
TOTAL		**10/10 (100%)**